⚡ DeepEval — The Open-Source LLM Evaluation Framework

"pytest for LLMs" — Unit-test your AI the same way you test your code

🔷 WHAT IS IT?

DeepEval is an open-source Python framework for evaluating LLM outputs. It provides 14+ research-backed metrics, integrates with pytest, supports CI/CD pipelines, and offers a cloud dashboard for monitoring.

🔶 WHY USE IT?

- LLMs can hallucinate, be toxic, or miss the point
- Human evaluation doesn't scale
- Need automated, reproducible scoring
- Catch regressions before production
- Evidence-based model selection

🔷 HOW DOES IT WORK?

1. Define LLMTestCase (input, output, context)
2. Choose metrics (Faithfulness, Relevancy…)
3. Run evaluate() or pytest
4. Get scores + reasons per metric
5. View results in CLI or cloud dashboard

⭐ 5,000+ GitHub Stars

📦 pip install deepeval

🐍 Python 3.8 — 3.12+

CORE PILLARS

🧪 Metrics
14+ LLM-as-Judge
+ custom G-Eval

🔗 Integrations
pytest · CI/CD
LangChain · LlamaIndex

📊 Dashboard
Confident AI cloud
Real-time monitoring

📁 Datasets
EvaluationDataset
Golden test cases

🛡️ Red-Teaming
Attack simulations
Vulnerability scans

🤖 Tracing
Agent + tool tracing
Multi-step observability

⚡ QUICK START (5 lines)

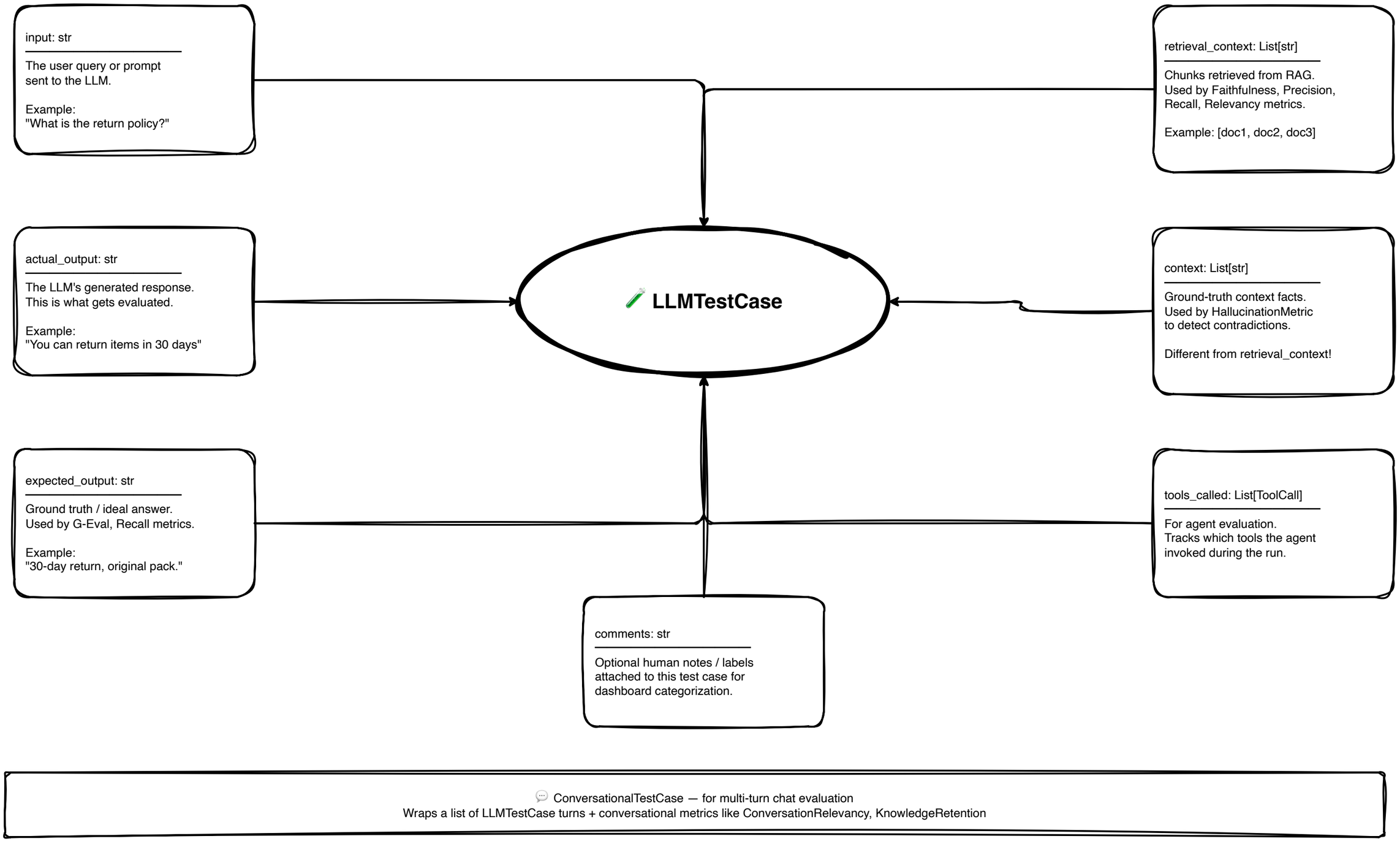
```
from deepeval import evaluate
from deepeval.test_case import LLMTestCase
from deepeval.metrics import AnswerRelevancyMetric

tc = LLMTestCase(input="What is RAG?", actual_output=my_llm_response, retrieval_context=[doc1, doc2])
evaluate(test_cases=[tc], metrics=[AnswerRelevancyMetric(threshold=0.7, model="gpt-4o")])
```

Created By : Ahmed
mahiahmad53@gmail.com
https://www.linkedin.com/in/ahmed-al-imaad

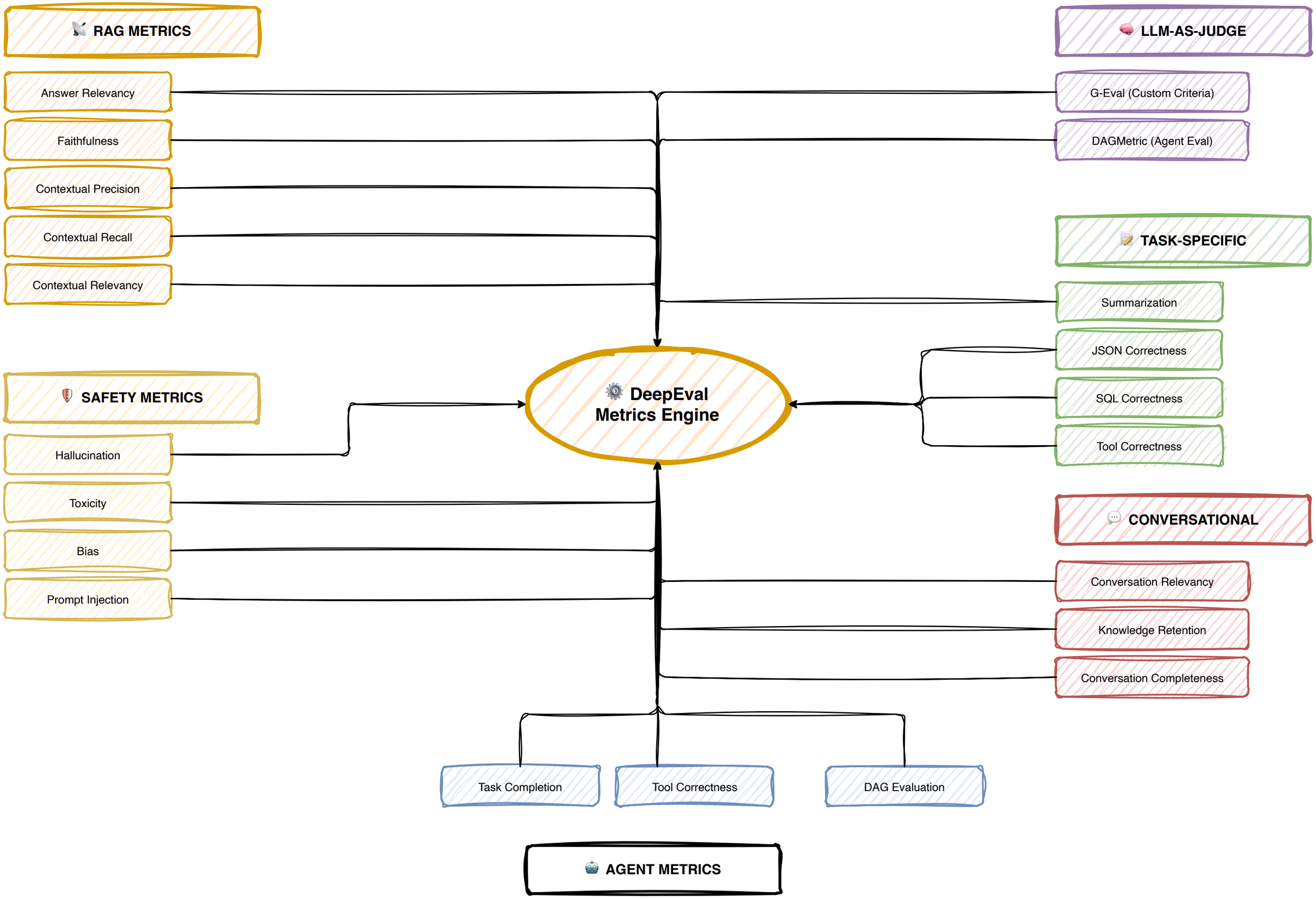
LLMTestCase — The Core Data Structure

Every DeepEval evaluation starts with an LLMTestCase — it holds all inputs and outputs for one test scenario



💬 ConversationalTestCase — for multi-turn chat evaluation
Wraps a list of LLMTestCase turns + conversational metrics like ConversationRelevancy, KnowledgeRetention

DeepEval — Complete Metrics Taxonomy



RAG Metrics — Deep Dive

🎯 Answer Relevancy

WHAT IT MEASURES:
Does the actual output directly address the input question?

USES: input + actual_output

HOW: LLM generates statements from output, then checks if each statement is relevant to the original question.
Score = relevant_statements / total_statements

THRESHOLD: 0.7 (recommended)
High score = answer stays on topic

🔍 Faithfulness

WHAT IT MEASURES:
Is the output grounded in the retrieved context? (No hallucination)

USES: actual_output + retrieval_context

HOW: Extracts claims from the output. For each claim, checks if it is supported by the retrieval_context.
Score = supported_claims / total_claims

THRESHOLD: 0.7
Low score = model making up facts not in context

📐 Contextual Precision

WHAT IT MEASURES:
Are the retrieved nodes relevant? Are relevant ones ranked first?

USES: input + expected_output + retrieval_context

HOW: Checks whether each retrieved chunk is relevant to the expected answer. Penalizes irrelevant chunks ranked highly.
Based on Average Precision (AP) formula.

THRESHOLD: 0.7
Low score = retriever returning noise

🔁 Contextual Recall

WHAT IT MEASURES:
Did the retriever find ALL necessary information?

USES: expected_output + retrieval_context

HOW: Breaks expected_output into sentences. For each sentence, checks if it is attributable to any retrieved chunk.
Score = covered_sentences / total_expected_sentences

THRESHOLD: 0.7
Low score = retriever missing critical documents

🧩 Contextual Relevancy

WHAT IT MEASURES:
How relevant is the retrieved context to the input question?

USES: input + retrieval_context

HOW: For each retrieved chunk, determines if it is relevant to answering the input. Similar to precision but without requiring expected_output.
Score = relevant_chunks / total_chunks

THRESHOLD: 0.7
Does not require ground truth labels

📋 Which RAG Metric To Use — Decision Guide

Fixing hallucinations?
→ Use Faithfulness

Answer off-topic?
→ Use Answer Relevancy

Retriever noisy?
→ Use Contextual Precision

Missing key docs?
→ Use Contextual Recall

No ground truth?
→ Use Contextual Relevancy

Full RAG audit?
→ Use ALL 5 metrics

Safety Metrics — Deep Dive

👻 Hallucination

WHAT IT MEASURES:
Does the output contradict the provided context facts?

USES: actual_output + context (NOT retrieval_context)

HOW: For each sentence in output, checks if it agrees with, contradicts, or is unrelated to the context facts.
Score = contradicted_facts / total_facts

⚠️ INVERTED METRIC: Lower score = better
THRESHOLD: 0.3 (fail if score EXCEEDS threshold)

KEY DIFFERENCE from Faithfulness:
Faithfulness uses retrieval_context (RAG chunks)
Hallucination uses context (ground-truth facts list)

☣️ Toxicity

WHAT IT MEASURES:
Does the output contain harmful, offensive, or toxic language?

USES: actual_output only

HOW: LLM evaluates whether the output contains any of:
- Hate speech / slurs
- Threats or violent language
- Harassment or bullying
- Sexually explicit content
- Dangerous misinformation

⚠️ INVERTED METRIC: Lower score = better
THRESHOLD: 0.1 (very strict for production)
Score = fraction of toxic content detected

⚖️ Bias

WHAT IT MEASURES:
Does the output show unfair bias toward any group?

USES: actual_output only

BIAS CATEGORIES DETECTED:
- Gender bias
- Racial / ethnic bias
- Religious bias
- Political bias
- Age / disability bias
- Socioeconomic bias

⚠️ INVERTED METRIC: Lower score = better
THRESHOLD: 0.5
HOW: LLM identifies biased opinions or stereotypes

💉 Prompt Injection

WHAT IT MEASURES:
Was the LLM successfully manipulated to ignore its instructions?

USES: input + actual_output

HOW: Detects whether adversarial instructions embedded in the input caused the LLM to deviate from its intended behavior.
Checks for jailbreaks, role-playing tricks, delimiter attacks.

⚠️ INVERTED METRIC: Lower score = better
THRESHOLD: 0.5
Critical for production safety guardrails

⚠️ Understanding Inverted Metrics
For safety metrics (Hallucination, Toxicity, Bias, Prompt Injection):
Score represents the AMOUNT OF BAD BEHAVIOUR detected.
So score=0.0 is PERFECT, score=1.0 is WORST.
is_successful() returns True when score < threshold (not >)

🔴 RED TEAMING — Extended Safety Suite
Attack Simulations built into DeepEval:
- Prompt Leakage • Jailbreaking • SQL Injection • SSRF • Shell Injection
- Intellectual Property • PII Leakage • Excessive Agency

Run via: red_team(target_llm, attacks=[...])

G-Eval — Build Any Custom Metric with LLM-as-Judge

🧠 What is G-Eval?

G-Eval lets you define any evaluation criteria in plain English.
DeepEval uses an LLM (GPT-4o, Claude, etc.) as the judge.
The model scores from 0.0–1.0 and provides a reason for the score.
Based on the NLP paper: "G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment"

1️⃣ **You write evaluation criteria in English** → 2️⃣ **DeepEval builds a structured prompt for the judge LLM** → 3️⃣ **LLM generates evaluation steps automatically** → 4️⃣ **LLM scores each step 1–5 using chain-of-thought** → 5️⃣ **Scores normalized to 0.0–1.0 with weighted avg** → 6️⃣ **Final score + reason returned to caller**

📌 evaluation_params — What Data Does G-Eval See?

INPUT	ACTUAL_OUTPUT	EXPECTED_OUTPUT	RETRIEVAL_CONTEXT	CONTEXT
The user question	LLM response	Ground truth	RAG chunks	Ground truth facts

💻 G-Eval Code Example — Custom Ecommerce Helpfulness Metric

```
from deepeval.metrics import GEval
from deepeval.test_case import LLMTestCaseParams

helpfulness = GEval(
name="EcommerceHelpfulness",
criteria="""Evaluate if the response is helpful for an ecommerce customer.
Award high scores when: (1) question is directly answered, (2) prices/steps/dates included,
(3) tone is professional and empathetic, (4) no follow-up required.""",
evaluation_params=[LLMTestCaseParams.INPUT, LLMTestCaseParams.ACTUAL_OUTPUT],
threshold=0.7,
model="gpt-4o", # Judge model
include_reason=True, # Returns explanation with score
)

# metric.score → float 0.0-1.0
# metric.reason → "The response answered the price question but missed the shipping timeline"
# metric.is_successful() → True if score >= threshold
```

Task-Specific Metrics

📝 Summarization

WHAT: Is the summary factually consistent with the source document AND does it cover the key information?

USES: input (source text) + actual_output (summary)

TWO COMPONENTS scored internally:
(1) Alignment Score — are claims in summary supported by source? (like Faithfulness)
(2) Coverage Score — are key facts from source mentioned in summary?

Final Score = min(alignment_score, coverage_score)
Both must be high for a good summary.

Code: SummarizationMetric(threshold=0.7, model="gpt-4o")

🔧 JSON Correctness

WHAT: Does the LLM output valid, correctly structured JSON matching a given schema?

USES: actual_output + expected_output (or schema)

VALIDATES:
• Is the output parseable JSON? (syntax check)
• Do all required keys exist?
• Are data types correct? (string vs int vs array)
• Are enum values respected?
• Does nesting match schema?

Score = correct_fields / total_required_fields

Use case: Tool-calling agents, structured data extraction

🗄️ SQL Correctness

WHAT: Is the LLM-generated SQL semantically equivalent to the expected query?

USES: actual_output (SQL string) + expected_output (correct SQL)

HOW: Uses an LLM to compare SQL queries for:
• Same tables / joins referenced
• Same filtering conditions (WHERE clauses)
• Same aggregations / GROUP BY
• Equivalent SELECT columns
• ORDER and LIMIT clauses

Does NOT require running against a database.
Use case: Text-to-SQL applications

🔨 Tool Correctness

WHAT: Did the agent call the right tools with the right parameters?

USES: tools_called (from LLMTestCase) + expected_tools

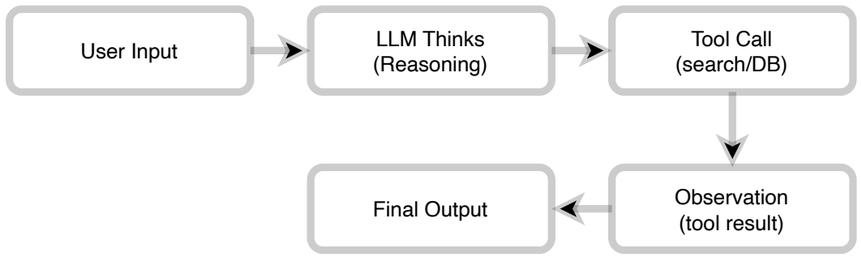
VALIDATES:
• Correct tool name called
• All required parameters passed
• Parameters have correct values
• Order of tool calls (if sequence matters)
• No extra/unnecessary tool calls

Score = correct_tool_calls / expected_tool_calls

Use case: ReAct agents, function-calling LLMs, AI assistants

Agent Evaluation + Conversational Metrics

🤖 How DeepEval Evaluates Agents



📊 DAGMetric — Directed Acyclic Graph Evaluation

Define expected agent behavior as a DAG:
- Each node = a reasoning step or tool call
- Edges = required sequence dependencies
- DeepEval checks if agent followed the DAG
- Supports branching (if/else logic in agent)
- Supports parallel tool calls

Use: TaskCompletionMetric, ToolCorrectnessMetric as nodes within a DAGMetric graph

💬 Conversational Evaluation

ConversationalTestCase:
A wrapper around multiple LLMTestCase turns.
Represents one full conversation (N turns).

Conversation Relevancy

Does each chatbot response address the user message in that turn?
Checks per-turn relevancy across conversation.

Knowledge Retention

Does the chatbot remember facts mentioned in earlier turns?
Detects context-window / memory failures.

Conversation Completeness

Did the chatbot fully resolve the user's goal across the conversation?
End-to-end task success check.

Role Adherence

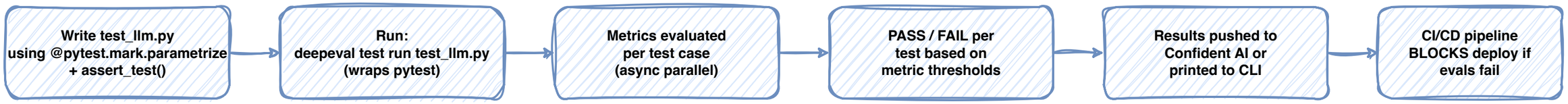
Does the chatbot stay in its defined persona / role throughout the chat?
Checks instruction-following across turns.

🔭 DeepEval Tracing — Observability for Agents

@observe decorator traces every LLM call, tool call, and retrieval step in your agent.
Sends trace data to Confident AI dashboard for latency, cost, and quality monitoring.
from deepeval.tracing import observe → @observe(type="llm") @observe(type="tool") @observe(type="retriever")

Created By : Ahmed
mahiahmad53@gmail.com
https://www.linkedin.com/in/ahmed-al-imaad

pytest Integration — LLM Evals in CI/CD

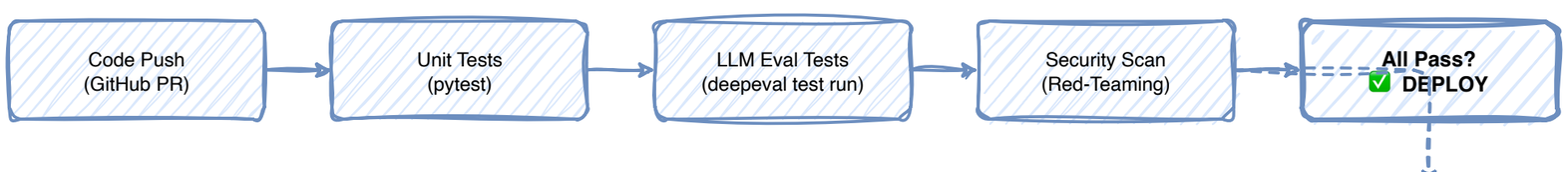


test_rag_chatbot.py — Full pytest Example

```
import pytest
from deepeval import assert_test
from deepeval.test_case import LLMTestCase
from deepeval.metrics import AnswerRelevancyMetric, FaithfulnessMetric

# Parametrize over multiple test inputs
@pytest.mark.parametrize("test_case", [
LLMTestCase(
input="What is your return policy?",
actual_output=my_rag_chatbot("What is your return policy?"),
retrieval_context=[retrieve_docs("return policy")],
expected_output="Items can be returned within 30 days in original packaging"
),
LLMTestCase(
input="How long does shipping take?",
actual_output=my_rag_chatbot("How long does shipping take?"),
retrieval_context=[retrieve_docs("shipping")],
),
])
def test_rag_system(test_case: LLMTestCase):
metrics = [
AnswerRelevancyMetric(threshold=0.7, model="gpt-4o"),
FaithfulnessMetric(threshold=0.7, model="gpt-4o"),
]
assert_test(test_case, metrics) # raises AssertionError if any metric fails
```

🔄 CI/CD Pipeline Integration



⚡ Performance: AsyncConfig(run_async=True, max_concurrent=20) — evaluates all test cases in parallel, reducing eval time from O(n) to near O(1)

Any Fail?
BLOCK

Confident AI Dashboard + Datasets + Synthesizer

📊 Confident AI Cloud Dashboard

📈 Real-time Metric Dashboard
Track scores over time per test suite
Spot regressions between model versions

🏷️ Test Case Annotation
Label and categorize test cases
Human feedback on outputs

🔍 Test Run History
Compare runs across Git commits
A/B testing different prompts

⚡ Live Monitoring
Production LLM call tracing
Latency + cost + quality metrics

```
deepeval login # authenticate
deepeval test run test_llm.py --identifier "v1.2"
# Results auto-push to dashboard
```

📦 EvaluationDataset + Golden

Golden = a saved test case template stored in Confident AI.
Has: input, expected_output, context, retrieval_context.
Does NOT have actual_output (generated at eval time).

Define Goldens (once, curated) → Push to Confident AI dataset.push(alias="v1") → Pull + generate actual_outputs at eval → Evaluate with DeepEval metrics

🧬 Synthesizer — Auto-Generate Test Cases from Documents

📄 Your Documents (PDFs, text, URLs) → 🧬 Synthesizer Generates Q&A pairs → 🎯 Evolve Questions (harder, multi-hop) → 📦 Golden Dataset (input + expected) → 🚀 Evaluate RAG System at Scale

```
from deepeval.synthesizer import Synthesizer
syn = Synthesizer(model="gpt-4o")
syn.generate_goldens_from_docs(document_paths=["docs/faq.pdf"], max_goldens_per_document=25)
dataset = syn.to_evaluation_dataset()
```

Evolution Types: SIMPLE → REASONING → MULTI-HOP → COMPARATIVE → HYPOTHETICAL
Each creates harder, more realistic test cases to stress-test RAG systems comprehensively.

Supported LLM Providers, Models and Configuration

🤖 Judge LLM Providers (model= parameter)

🟢 **OpenAI**
gpt-4o
gpt-4o-mini
gpt-4-turbo
o1-preview

🔵 **Anthropic**
claude-3-5-sonnet
claude-3-opus
claude-3-haiku
claude-sonnet-4

🟣 **Azure OpenAI**
Hosted on Azure
Same models as OAI
Enterprise SLA

🟡 **Google Vertex**
gemini-1.5-pro
gemini-1.5-flash
gemini-2.0-flash

🦙 **Local / Ollama**
llama3, mistral
Any Ollama model
Zero API cost

⚙️ **Custom DeepEvalLLM**
Wrap any LLM API
Extend BaseMetric
Full control

⚙️ AsyncConfig — Control Parallel Evaluation

```
AsyncConfig(
run_async=True, # Enable parallel eval (default True)
throttle_value=0, # Seconds between API calls (rate limiting)
max_concurrent=20 # Max parallel metric evaluations
)

# With run_async=True: 5 test cases × 10 metrics = 50 LLM calls run in parallel
# throttle_value=1: adds 1s delay between calls to avoid rate limits
```

💾 CacheConfig — Save API Costs

```
CacheConfig(
write_cache=True, # Cache LLM judge calls locally (.deepeval cache)
use_cache=True # Re-use cached results for identical inputs
)

# write_cache=True: First run saves judge LLM outputs to local cache
# use_cache=True: Re-runs skip API calls for unchanged test cases
# Reduces cost by ~80% on re-runs of the same dataset
```

🚨 ErrorConfig

```
ErrorConfig(
ignore_errors=False, # Raise on metric error
skip_on_missing_params=False # Fail if params missing
)
```

🖥️ DisplayConfig

```
DisplayConfig(
print_results=True, # CLI table output
verbose_mode=True, # Step-by-step logs
show_indicator=True # Progress bar
)
```

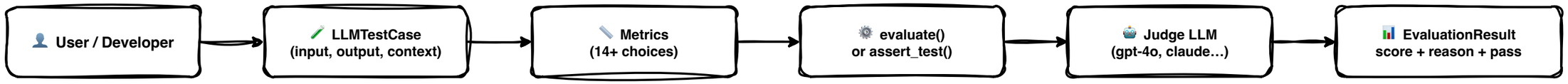
🛠️ Custom Judge LLM — Bring Your Own Model

```
from deepeval.models import DeepEvalBaseLLM

class MyJudge(DeepEvalBaseLLM):
def load_model(self): return my_model
def generate(self, prompt: str) -> str: return my_model.predict(prompt)
def get_model_name(self) -> str: return "my-judge-v1"

# Use: AnswerRelevancyMetric(model=MyJudge())
```

DeepEval — Full Architecture + Complete Cheat Sheet



COMPLETE METRIC CHEAT SHEET

Metric	Uses	Measures	Default Threshold	Inverted?	Category
Answer Relevancy	input, actual_output	Is answer on-topic and relevant to the question?	0.7	No	RAG
Faithfulness	actual_output, retrieval_context	Are output claims grounded in retrieved context?	0.7	No	RAG
Contextual Precision	input, expected_output, retrieval_context	Are relevant retrieved chunks ranked highest?	0.7	No	RAG
Contextual Recall	expected_output, retrieval_context	Did retriever find all info needed for expected answer?	0.7	No	RAG
Contextual Relevancy	input, retrieval_context	Are retrieved chunks relevant to the question? (no GT needed)	0.7	No	RAG
Hallucination	actual_output, context	Does output contradict ground-truth context facts?	0.3	**YES** ⚠️	Safety
Toxicity	actual_output	Is output harmful, hateful or offensive language?	0.1	**YES** ⚠️	Safety
Bias	actual_output	Does output show unfair gender/race/political bias?	0.5	**YES** ⚠️	Safety
G-Eval	any (you choose)	Any custom criteria written in plain English (flexible)	0.7	No	Custom
Summarization	input, actual_output	Is summary factually aligned AND covers key info?	0.7	No	Task
Tool Correctness	tools_called	Did agent call right tools with correct parameters?	1.0	No	Agent
Prompt Injection	input, actual_output	Was LLM successfully manipulated by adversarial prompt?	0.5	**YES** ⚠️	Safety

⚡ **DeepEval by Confident AI — docs.confident-ai.com | pip install deepeval | deepeval login | deepeval test run test_llm.py**

Created By : Ahmed
mahiahmad53@gmail.com
https://www.linkedin.com/in/ahmed-al-imaad